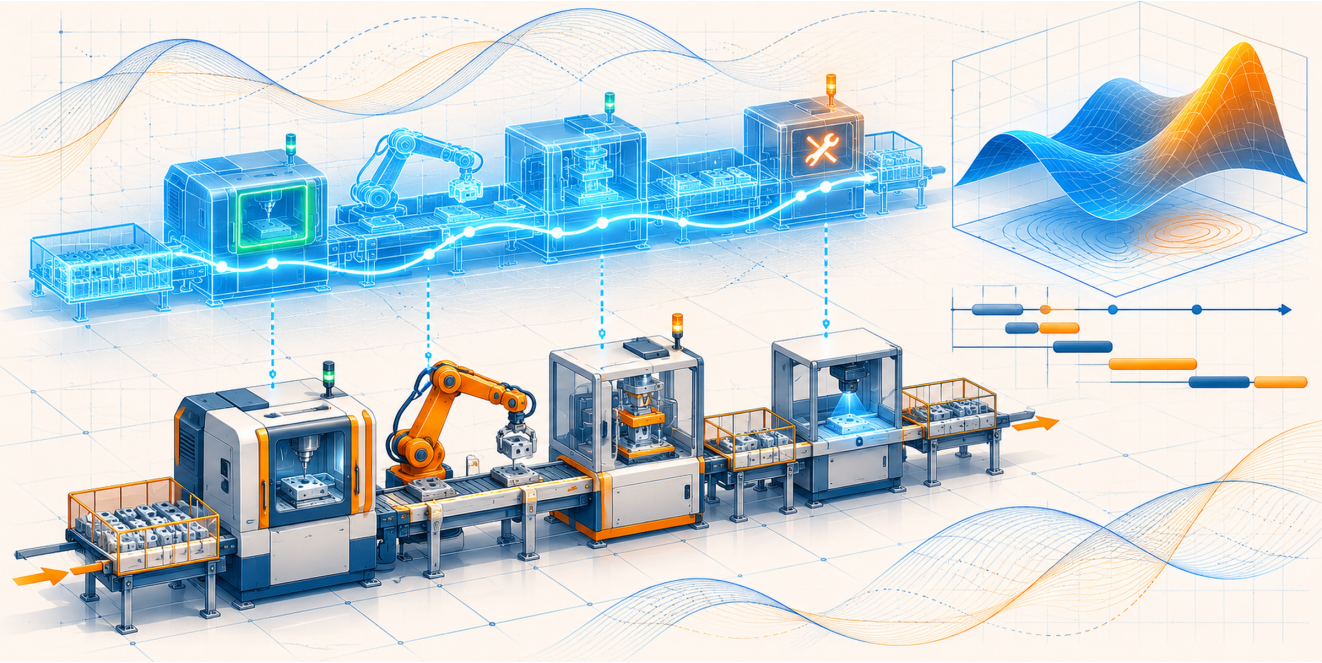
Factory-Line Digital Twin Formula Modules with paceval

Reusable mathematical graphs inside a finite-buffer discrete-event manufacturing study



Concept artwork only. The figure is not a photograph of a real plant; numbered scientific figures later in the report derive from executed CSV results.

EXECUTED SCOPE	VERIFIED RESULT
Persistent paceval mathematical functions	38 graphs, including 20 interchangeable local process-time versions
Manufacturing study	20 simulations, 440 synthetic incoming jobs each, four serial machines
Actual paceval batch calculation	209,798 outputs through 98 batch invocations
Independent checks	16,302 native and 1,122 high-precision reference values

Research status. A successfully executed mathematical and deterministic event-simulation feasibility study. No real plant data, industrial commissioning, predictive fidelity or measured speed advantage is claimed.

1. Introduction and industrial use case

Production-line simulation must explain how arrival streams, individual machining operations, machine downtime, intermediate buffers, scrap and bottlenecks interact. A local cycle-time formula is necessary but insufficient: the same expression can produce different system throughput when an upstream station is blocked or downstream capacity is unavailable.

The source proposal [1] isolates the simplified relationship **throughput = rate × (1 − downtime fraction)** and proposes a public synthetic event trace with twenty interchangeable expressions. WP_040 extends this proposal into a reproducible four-station digital-twin research demonstrator. The essential question is whether versioned mathematical modules can be evaluated by the original paceval engine and used without changing the host simulation semantics.

Scientific and industrial context

NIST describes ISO 23247 as a reference framework for manufacturing digital twins [2,3]. The newly published ISO 23247-6:2026 addresses composition and interoperability among manufacturing twins [4]. The FMI 3.0 Implementers' Guide provides additional guidance for tool and model interoperability [5]. These references inform module separation, but the present program does not claim compliance, FMU export or a real-time bidirectional industrial twin.

What the demonstrator can and cannot answer

RESEARCH QUESTION	DEMONSTRATED EVIDENCE
Can a local formula be replaced?	20 separately registered persistent cycle-time expressions; same event-host code
Can mathematical results propagate to a line?	Per-operation paceval durations feed event scheduling, finite queues and final throughput
Can manufacturing KPIs use paceval?	18 additional persistent graphs include quality, energy and operational metric formulas
Can field performance be certified?	Not from this experiment: no measured factory data or calibrated failures

The experiment is intentionally a numerical manufacturing-system kernel, not a general-purpose shop-floor integration platform.

2. Defined four-station event-driven model

The production system represents a deterministic serial process with the stations **Cutting → Robot transfer → Inspection → Packaging**. Every incoming unit has a unique job identifier and a scheduled exogenous arrival. Each station has one processing resource and FIFO input; downstream waiting buffers have capacities 3, 4 and 3. The external station-one backlog is unbounded and reported separately.

ELEMENT	EXACT RESEARCH ASSUMPTION
Incoming trace	440 jobs; arrival times in minutes: 4.80 × job + 0.14 × (job mod 7)
Nominal station times	4.15, 5.15, 4.55, 4.80 min per operation, respectively
Fixed intermediate buffers	Before stations two, three, four: 3, 4, 3 waiting slots
Temporary shutdowns	Two predetermined downtime windows per station, persisted in C++ code
Work conservation	Each arrival finishes as a passed unit or is rejected at a completed station
Quality outcomes	Fixed pseudo-random draws compared with paceval mathematical reject probabilities
Event scheduling	C++ priority queue; completed upstream stations block if next FIFO buffer is full

Model structure and validation requirements

A machine’s processing completion time is propagated through fixed unavailable windows without performing work during downtime. The simulated duration remains the active machine time predicted by paceval. A completed but blocked upstream station cannot start the following job until the next buffer has room.

The output guard requires every input job to leave the line exactly once, no overlapping operations at the same station, no downstream operation beginning before the previous station completes, and no negative processing time. These invariants are checked independently from stored event CSVs.

3. Mathematical models and units

All twenty local formula versions are defined on synthetic operating attributes: b (nominal minutes/operation), w (wear proxy in [0,1]), T (temperature in degrees Celsius), L (load proxy) and φ (deterministic phase). The index k ranges from 0 to 19. The complete literal formula strings are included in the executed registry and are never silently reconstructed by Python for scientific plots.

$$t_{k,s,j} = b_s\Big[1 + \beta_k + 0.011 w_j + 0.0009(T_j - 22)^2 + 0.014 L_j + \alpha_k \sin(\varphi_j + 0.09k) + \delta_k w_j L_j\Big]$$

$$\beta_k = 0.010(k - 9.5), \quad \alpha_k = 0.003((k \bmod 4) + 1), \quad \delta_k = 0.006((k \bmod 5) - 2)$$

A time result from this function is used as the station service requirement in the actual event calendar. It can be precomputed because the source job-specific wear, temperature, load and phase are exogenous, not derived from mutable queue state. Consequently the batch path and the event simulator use exactly the same frozen input records.

Additional quality and energy formulas

$$p_{\text{reject}} = 0.006 + 0.010 w + 0.00007(T - 22)^2$$

$$E_{\text{operation}} = 0.018 b(1 + 0.2L) \quad [\text{kWh}]$$

Quality probability is a declared uncalibrated surrogate, not measured failure propensity. Electricity is a declared synthetic per-operation model, not meter data. The same random draws and the same quality/energy graph handles are used in all twenty scenarios to isolate the mathematical effect of replacing cycle-time formulas.

4. Operational mathematics executed by paceval

After simulating each scenario, C++ passes aggregate observed event counts and durations to further persistent mathematical graphs. Those graphs, rather than a hidden native replacement, calculate the reported final numerical KPIs.

$$R_{\text{good}} = 60 N_{\text{good}} / T_{\text{line}} \ [\text{units/hour}]$$

$$Y = N_{\text{good}} / N_{\text{arrived}}, \qquad C_{\text{electricity}} = p_E E_{\text{total}}$$

The study defines a station-two **OEE-like decomposition** solely for illustrating reusable mathematical modules: (busy/T) × (ideal × processed/busy) × (passed/processed). The terms correspond respectively to active-time fraction, ideal-cycle performance and station first-pass quality. They are not a certified or standardized industrial OEE measurement.

FUNCTION GROUP	ENGINE RESPONSIBILITY
20 cycle-time variants	Twenty independently stored paceval computation handles, evaluated in multi-graph batches
Quality, electricity and local diagnostics	Six real mathematical graph handles, including reject probability, energy and cycle-time comparisons
System and station KPIs	Twelve additional real graph handles, including throughput, yield, OEE-like metrics, power-cost proxy and target margin
Stateful events and decisions	C++ host only: event calendar, queues, machine blocking, deterministic shutdowns, random draws and file I/O

The complete expression texts, API assignments and ownership proof are in `examples/formula_registry.csv` and `results/math_execution_inventory.csv`. The batch API works on many input vectors and many formula handles per invocation.

5. Actual original paceval SDK execution

The C++ program compiles against the original `paceval_main.cpp` and `paceval_main.h` interface files and loads the separately supplied compatible original Linux paceval runtime. Every expression is registered once using `paceval_CreateComputation`. The production workload uses `paceval_dGetMultipleComputationsResultsExt` for the precomputed four-station operation matrix, the numerical response surface and the final system KPI evaluations.

RECORDED EXECUTION FACT	ACTUAL RESULT
Registered original paceval graphs	38
Distinct replaceable process models	20
Full operation-input rows	1,760 (440 jobs × 4 stations)
Full simulated scenarios	20, with 440 arrivals per scenario
Multi-graph batch invocations	98
Individually checked batch outputs	209,798
Independent native mathematics checks	16,302
Independent high-precision mathematics checks	1,122 at 80 decimal digits

Error handling and reference checks

Every per-result SDK error code is checked. The tested Linux SDK’s multi-graph call reports `false` for success; treating that return value as a conventional `true = success` indicator would be incorrect. Both finite outputs and independently programmed native mathematical formula values are tested. The numerical reference calculations do not replace the original paceval outputs.

The generated response surface is 61 × 61 operating points with declared wear and temperature axes. It is a real paceval calculation and does not represent measured workcell behavior.

6. Full reproducibility and numerical audit

The code and deterministic source trace establish complete provenance. Per-operation paceval values are written to CSV, then inserted without reinterpretation into the event simulator. Every run uses the same seeded quality decisions. Original expression strings are additionally published for independent review.

INDEPENDENT GATE	RESULT
CMake compilation and full CTest SDK execution	Passed
C++ independent formula checks	16,302 passed
80-decimal-digit mpmath samples	1,122 passed
Max sampled native normalized difference	1.6471195827390907e-16
Arrival/output conservation	440 arrivals = good + scrap for each scenario
Baseline machine overlaps	None: four station calendars checked from exported events
Baseline stage chronology	Every visited downstream stage begins after upstream completion

Determinism versus physical certainty

Reproducing the exact same CSV proves deterministic software execution under the same fixed inputs, not that factory machines will produce those event trajectories. The study treats real sensor-derived wear, machine-specific downtime distributions, empirical scrap probability and observed power as separate, future calibration tasks.

The independently audited output is a complete mathematical/discrete-event numerical test. Independent metrology, plant telemetry and measurement uncertainty are not included.

7. Scientific figures I: event progression and rate

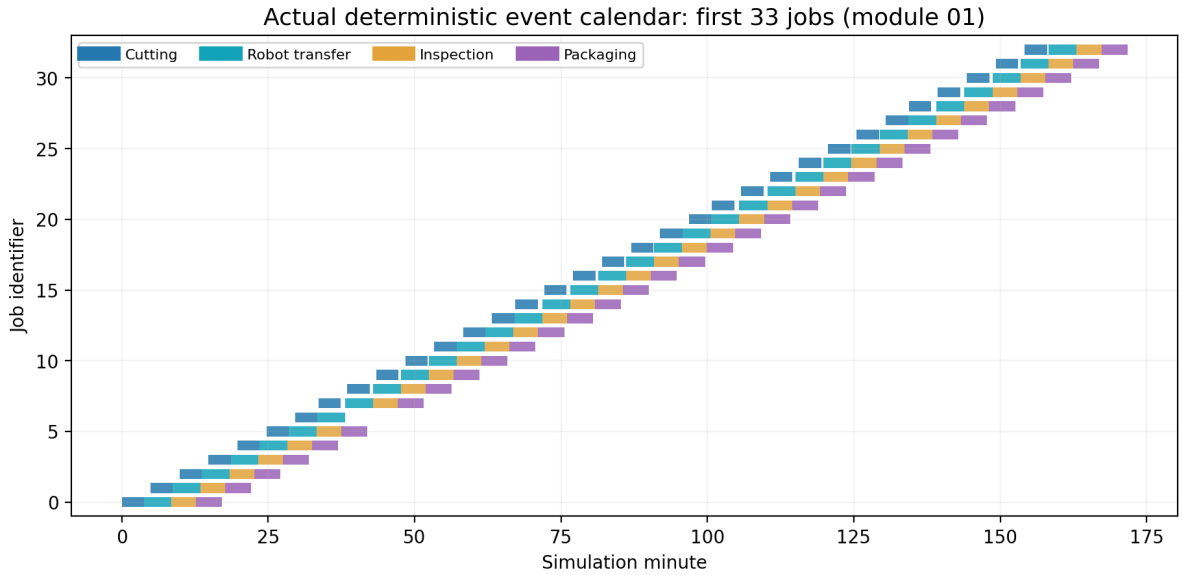


Figure 1. Scheduled station processing and event-calendar completion of the first 33 jobs of model 01. Longer calendar intervals can include declared downtime.

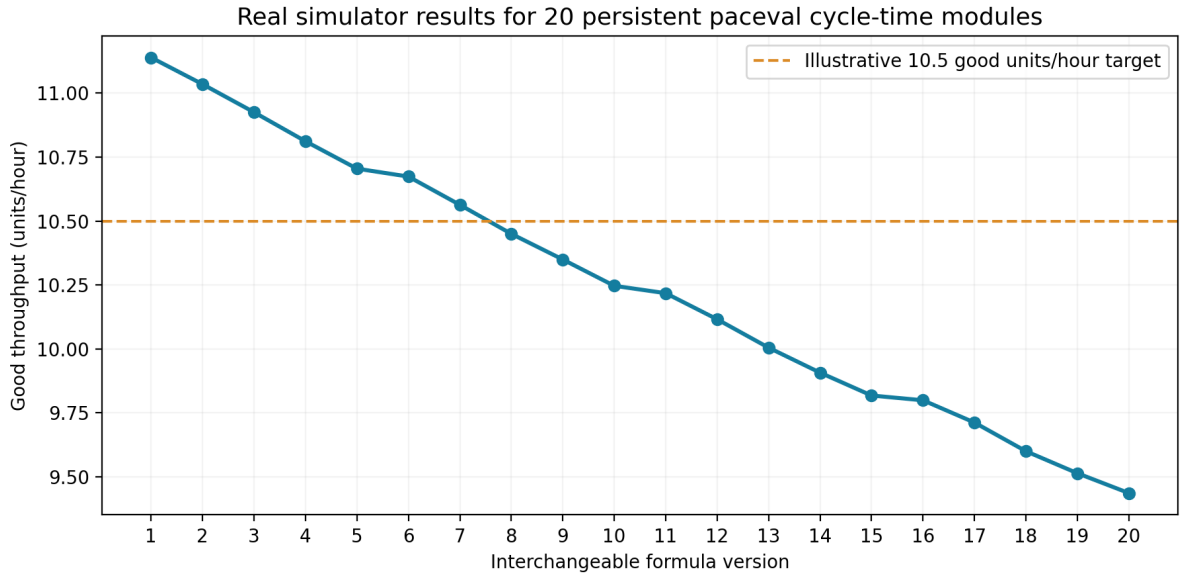


Figure 2. Executed good-unit throughput for all twenty independently registered and reused mathematical formula versions; line inputs and quality draws are held fixed.

8. Scientific figures II: mathematical surfaces

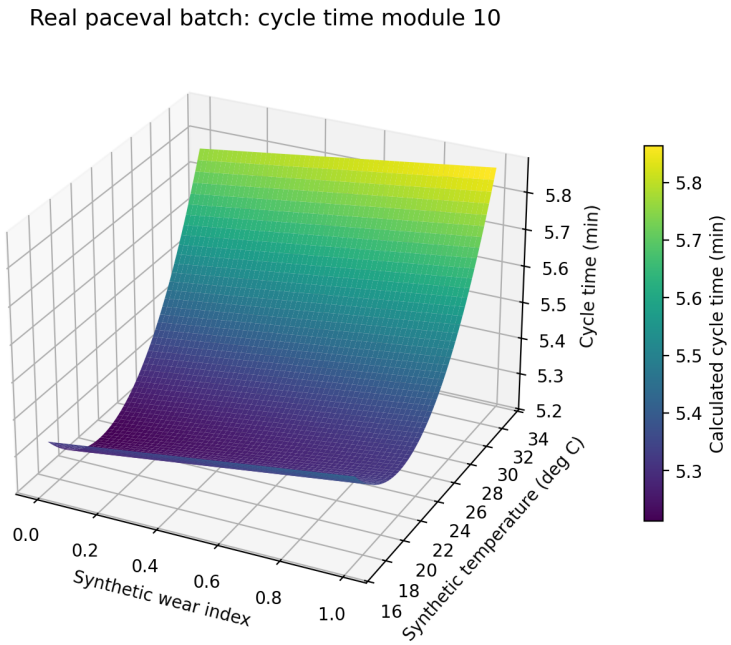


Figure 3. Actual 61 × 61 paceval computation for process time from model 10 over the wear-temperature domain.

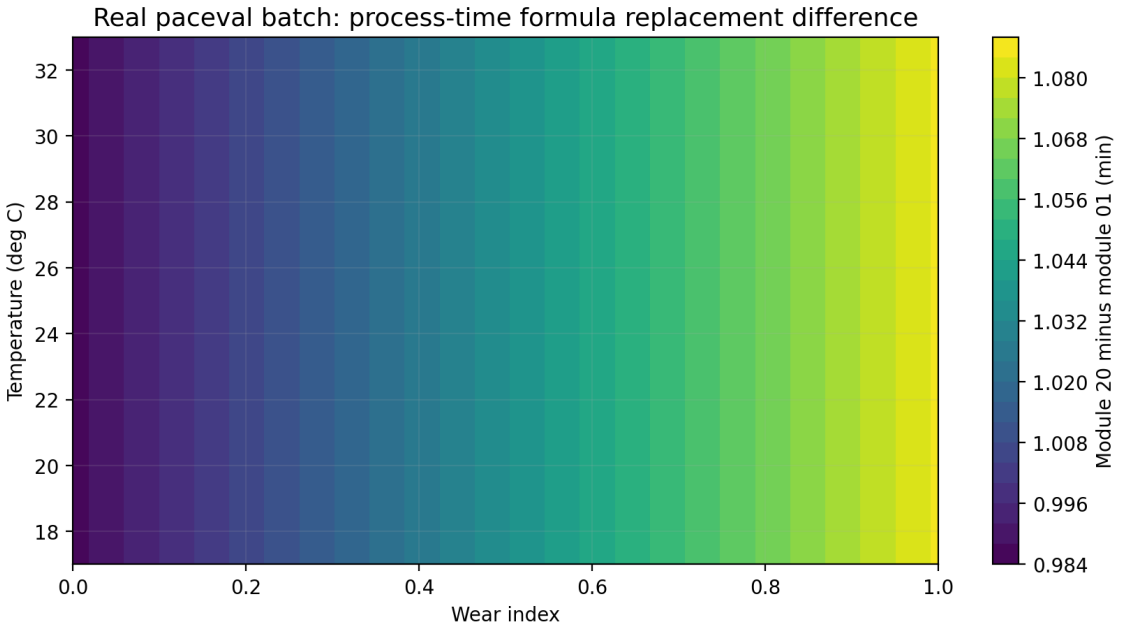


Figure 4. The mathematical difference in predicted cycle time between independent models 20 and 01, computed through paceval.

9. Scientific figures III: system behavior

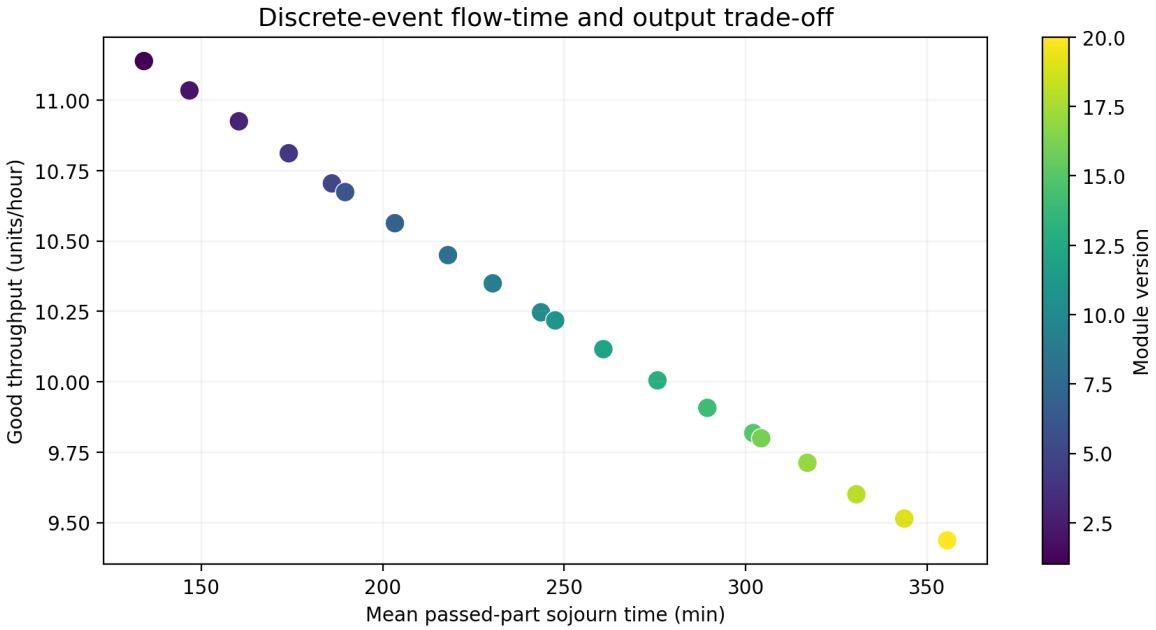


Figure 5. Line good-unit output versus the observed mean flow time of passed parts, where buffer blocking and deterministic shutdowns influence completion order.

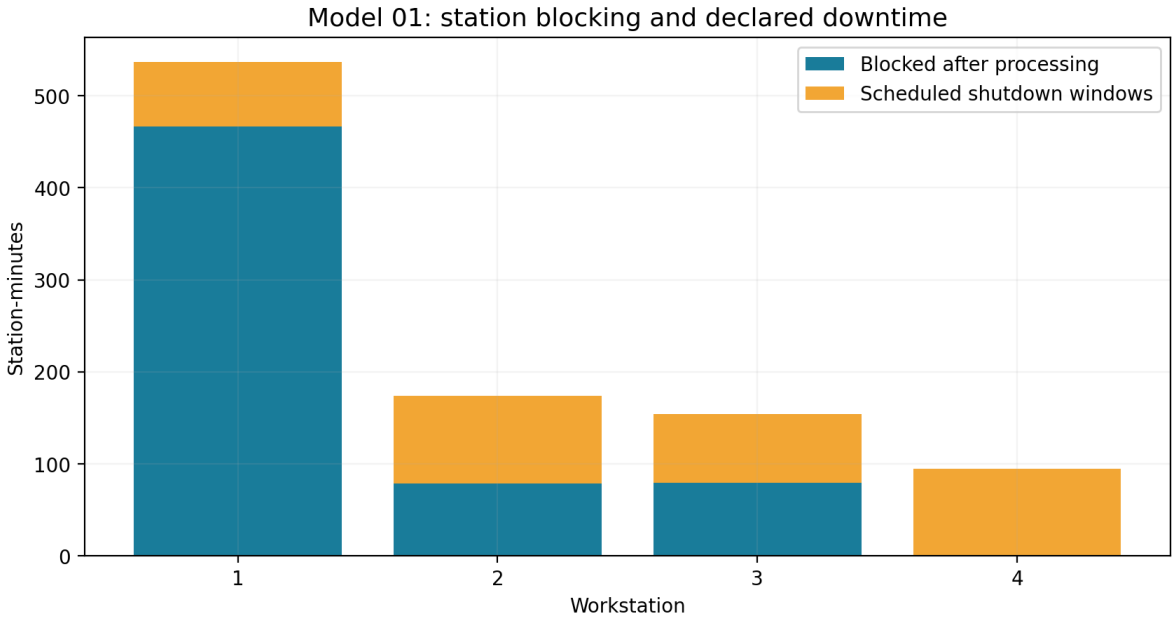


Figure 6. The source of lost station-minutes in scenario 01: recorded after-service blocking and scheduled downtime are different mechanisms.

10. Scientific figures IV: KPIs and conservation

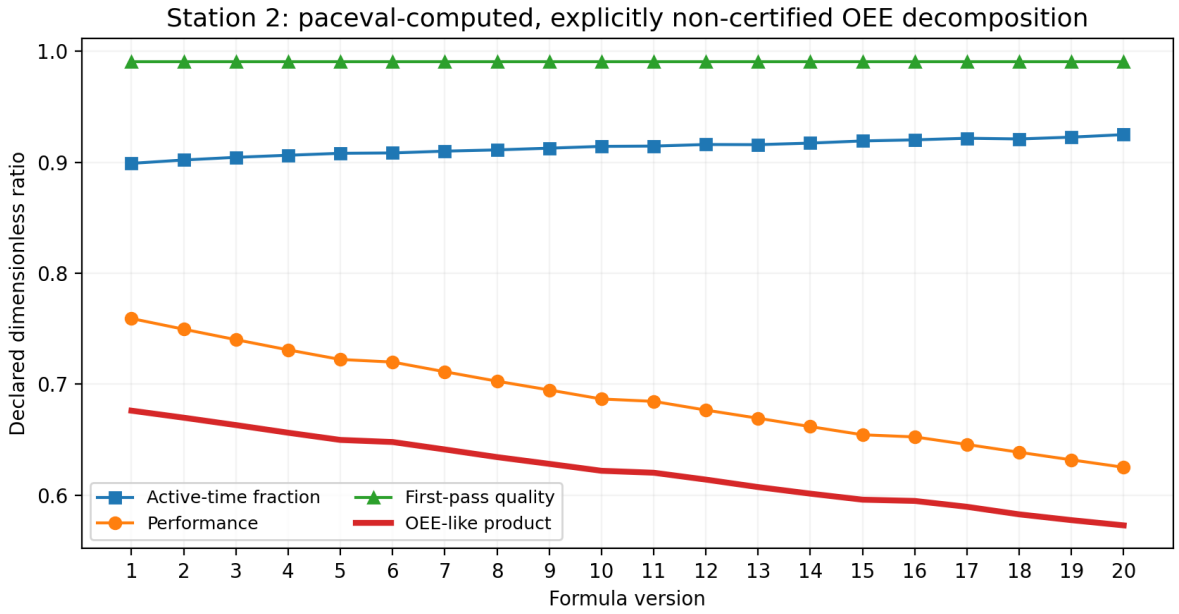


Figure 7. Original paceval stage-two KPI outputs under a deliberately defined, non-certified OEE-style decomposition.

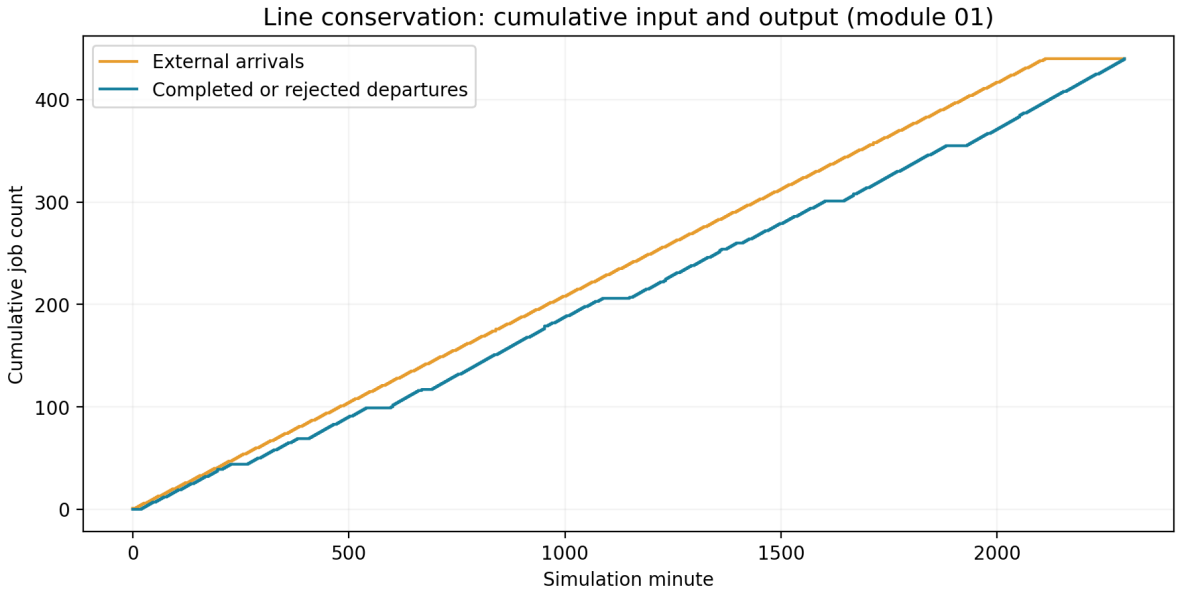


Figure 8. All 440 arrivals and all final good/scrap exits represented in the same deterministic event-calendar trace.

11. Scientific figures V: formula execution and controls

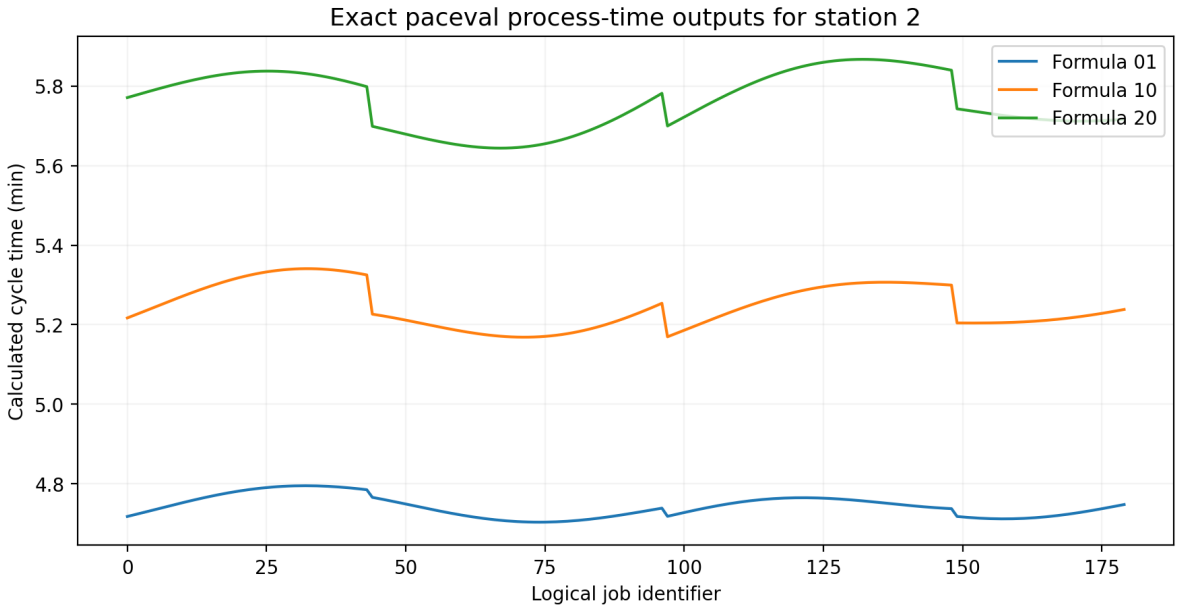


Figure 9. Literal paceval cycle-time graph values for the same station and the same input job index, with three different independently retained versions.

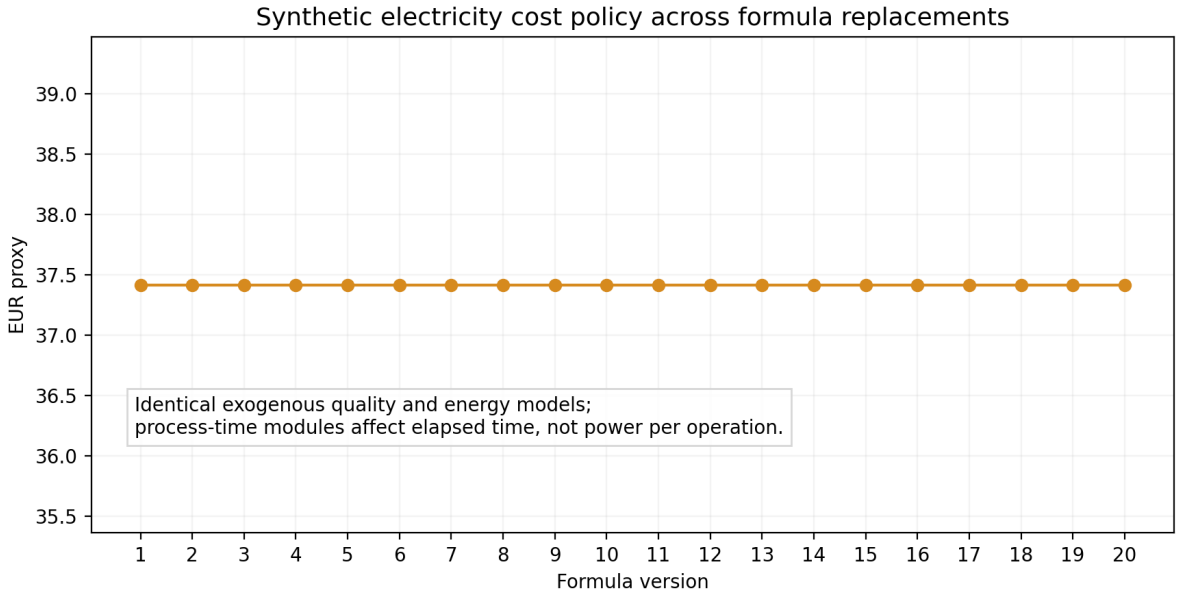


Figure 10. Electricity cost remains unchanged when the versioned process-time function is changed because the isolated per-operation energy and quality definitions are held fixed.

12. Scientific interpretation and limitations

The decisive result is the successful integration of twenty interchangeable, actual paceval mathematical functions into a working finite-buffer discrete-event manufacturing simulator. Formula replacement changes process durations and therefore line waiting times and final throughput. The simulator’s stateful logic is intentionally hosted by C++, not misrepresented as one giant mathematical expression.

What has NOT been demonstrated

• No physical factory-line measurements, actual sensor connection, machine parameter fit, real breakdown distribution, realistic mixed-product scheduling or calibration dataset.

• No hardware energy measurements, no cost savings from actual equipment, no measured performance speedups, no certified industrial OEE calculation.

• No FMI 3.0 executable FMU, ISO 23247 conformance, standardized factory digital-twin composition, or validated digital shadow.

• No mathematical guarantee that any one of the twenty synthetic formulas reflects a specific machine, plant, workcell or failure process.

Engineering implications

The approach is relevant where many expressions must be maintained or compared while preserving a stable, auditable event host. C++ can provide data ingestion, event priority, decisions and state; paceval provides deterministic functional evaluation and reusable expression lifecycle. The performance advantage, if any, requires a separately controlled timing study, not a theoretical claim from formula counts alone.

All conclusions above are limited to the frozen synthetic experimental design and its reproducible outputs. The source proposal remains a research hypothesis; the new project provides a verified implementation within that boundary.

13. Reproduction, references and research outlook

One-command reproducibility after obtaining the SDK runtime

Configure from `src/CMakeLists.txt` with `-DPACEVAL_RUNTIME=/absolute/path/to/libpaceval_linux_sharedLIB.so`, build with CMake, run CTest, then run `./build/factory_line_twin --out . --jobs 440 --variants 20 --surface 61`. Python scripts under `src/` independently verify the CSV values, produce the scientific figures and generate this PDF.

Recommended follow-up validation

• Collect real machine transition logs and workpiece-level state records; define exact engineering units, time synchronization and line topology.

• Determine empirical distributions of breakdown duration, repair, scrap, energy and queue routing; calibrate independently from an evaluation holdout.

• Validate source-by-source process-time expressions and downstream throughput on measured shift intervals with uncertainty quantification.

• Investigate OPC UA / MTConnect integration, standard digital-thread provenance, and FMI or similar component interfaces if a co-simulation system is needed.

References

[1] User-supplied independent proposal: WP_040 Factory-Line Digital Twin Formula Modules, November 2026.

[2] NIST (2021): Use Case Scenarios for Digital Twin Implementation Based on ISO 23247. https://doi.org/10.6028/NIST.AMS.400-2

[3] NIST (2026 update): Digital Twins for Advanced Manufacturing. https://www.nist.gov/programs-projects/digital-twins-advanced-manufacturing

[4] ISO 23247-6:2026, Digital twin composition. https://www.iso.org/standard/87426.html

[5] Modelica Association, FMI 3.0 Implementers Guide (2026). https://modelica.github.io/fmi-guides/main/fmi-guide/

[6] paceval original Mathematical Engine API. https://paceval.com/api/

Conclusion

Verified: a reproducible, original-paceval C++ mathematical-execution architecture for a synthetic event-driven factory line. **Not claimed:** validated plant digital-twin fidelity, deployed integration, industrial performance benefit or a measured return on investment.